

School of Electronic Engineering
and Computer Science

FINAL REPORT

Programme of study:
COMPUTER SCIENCE

Project Title:
**PROGRAMMING BY
DEMONSTRATION OF
HUMAN DECISION IN
A HUMAN-ROBOT
COLLABORATIVE
SORTING TASK**

Final Year
Undergraduate Project 2019/20

Supervisor:
LORENZO JAMONE

Student Name:
YUHANG ZHANG



Date: 11th MAY 2019

Acknowledgements

I would like to thank my supervisor, Dr Lorenzo Jamone, for guiding me and delivering vital feedback on my final year project. The scheduled meetings motivate me during the progress of this project.

Abstract

This paper proposed a system in ROS for making a pick-and-place using the Universal Robot Arm (UR5), and the system makes the robot being able to learn from human demonstration and perform sorting task using robot visualisation. This paper covers robotic topics, including Programming by Demonstration (PbD), and Human-Robot Collaboration (HRC), where these topics are taking advantage of the computer vision in ROS distribution using OpenCV to capture data from the camera sensor. The robot visualisation applies object identifying and tracking by 2D codes contributed by ARUCO markers. The paper also investigates the task planning and modelling by using Finite State Machine (FSM) in order to perform a human-robot shared task that is oriented by a user's demonstration. In particular, this paper takes into account of SMACH library of ROS for task planning and task manipulating. Where the challenge of task modelling is making the designed automation for the robot performing complex shared task, and being able to handle different sorting sequence when new user demonstration comes, but always doing the sorting in uniform distributed sequences.

C contents

Chapter 1: Introduction	1
1.1 Background.....	1
1.2 Problem Statement.....	2
1.3 Aim.....	3
1.4 Objectives	4
1.5 Research Questions	5
Chapter 2: Literature Review.....	6
2.1 Field of Study.....	6
2.2 Related Works	9
Chapter 3: Requirements	11
3.1 Functional and Non-Functional Requirements.....	11
3.2 Hardware	13
3.3 Software.....	14
Chapter 4: Task Model	15
4.1 Collaborative Sorting Task	15
4.2 State Machine.....	17
Chapter 5: Implementation	20
5.1 Computer Vision	20
5.2 Robot Control	23
5.3 Task Manager.....	24
Chapter 6: Presentation.....	25
6.1 Fragment of Code for SMACH	25
Chapter 7: Evaluation	26
7.1 System Findings	26
7.2 Comparison to the Gaps	32

7.3	Limitations.....	33
Chapter 8:	Conclusion	34
8.1	Achievements and Challenges	34
8.2	Future work.....	36
Chapter 9:	Risk Assessment	37
References		39
Appendixes		41
Project Plan		41
Milestones		41

Chapter 1: Introduction

1.1 Background

The vision for Human-robot collaboration (HRC) is to make robots a trustworthy team member. Many robots like the UR5 are being developed to help us perform better through collaborative tasks. The advantage of Programming by Demonstration (PbD) is to allow robots to perform different tasks, and not be limited to just performing one repetitive task. It can be complicated to program a robot to work in a flexible working environment. PbD is a common way of recording the reusable actions of human through user demonstration. This can allow robots to understand and imitate human demonstrations, such as keen decision-making ability which robots can efficiently imitate through PbD.

The design of this project is to equip an industrial robotic arm (UR5) with Robot Vision technology via an RGB-D camera to capture objects and movements. This would allow us to develop an autonomous system to perform a collaborative shared task with a human. This robot automation could be done by inputting a state transition model in the form of FSM (Finite State Machine), which allows the robotic arm to understand the sequence of the task it needs to execute. The robotic arm would be able to then schedule a specific task corresponding to human demonstration, therefore, allowing the robot to handle all the different possible executions the human partner may perform.

This paper uses object-oriented recognition by labelling objects with unique markers sorted by types, allowing the robot to detect the location and track the movement of objects. The actual task model will be programmed manually and given in advance to the robotic arm so that the robot will be able to execute and understand the sequence of collaborative tasks. The FSM task model includes every state the robot may act during the task and contains the possible transitions from the current state. This project views the human-robot collaborative sorting task as an attempt of PbD, where the robotic arm help humans to sort the numbers of cubes that are marked with ARUCO into specified subareas on a tabletop (where we use ARUCO boards on the working platform). The task executed by the robotic arm is based on a pick-and-place motion, a typical robotic

arm application. The ARUCO marker is considered as an effective approach for the robot to recognize objects in 2D dimensional space.

1.2 Problem Statement

Robots are currently widespread in our lives and play a vital role in many aspects, especially in industrials and social services. Robots can do better than humans in many jobs with higher accuracy, efficiency, and a lower rate of mistake. For instance, a cleaner robot can repeat its motions constantly to achieve its task. Such robots with complex algorithms would probably never miss cleaning a corner. However, the automation of a robot is costly due to high investments during its development stage. Shifting to the industrial practices, high cost of having robotic automation and the time-consuming of robot re-programming are still making the use of robot applications very narrow having taking consideration of the Smaller and Medium-sized Enterprises (SMEs). SMEs need robots that can use 'more frequent set-ups and re-programming of their robotic workers to adapt quickly for production of new products or variants of existing ones' (PricewaterhouseCoopers, 2014). IFR (2019) states that 'the cost of software and systems integration accounts for between 60 to 75% of the total cost of robot ownership' (p.11). This paper proposes a solution to increase the efficiency in software development through PbD, making the use of robots uncomplicated in complicated sequential tasks.

1.3 Aim

The project aims to implement a system that allows the robotic arm to learn from human demonstrations and assist in a Human-Robot collaboration sorting task. By using an external camera sensor, the system can recognize human demonstrations by tracking objects. The idea of the system is to use the concept of PbD to control the robot that performs the task without programming it. In addition, the system will be developed through natural programming on Kinetic ROS distribution. Finally, the project will use the Gazebo simulator and then use a real UR5 robotic arm to perform the experiment on the system with shared sequencing tasks.

1.4 Objectives

1.4.1 To Construct the State Transition Model

The main objective of this paper is to construct a proper and efficient state transition model to enable the robot in understanding the design task beforehand. This would therefore allow robots to collaborate with human in all possible situations of work flows. The transition model will be demonstrated and created in a type of FSM for early modelling on paper and will later on be programmed in ROS package by using SMATCH python library. This will be investigated in detail later in Chapter 3.

1.4.2 To Construct ARUCO Marker Image Recognition Algorithms Adapted with ROS

In order to visualize the data from the camera sensor, this paper uses ARUCO marker to make the detecting and tracking more efficient. The image recognition algorithm of the ARUCO marker will be done by using marker and OpenCV library of python in ROS.

1.4.3 To Have Experimental Study with ROS Programming and Industrial Robots

Another very important objective of this paper is to get familiar with ROS (Robot Operating System), where it will be working on controlling the industrial robotic arm (UR5) through programming over the ROS and building communications between each layer. This becomes the fundamental investigation of this project, that is, being able to program a proposed system in a ROS package for performing complex tasks.

1.4.4 To Have Some Investigation on Particular PbD, HRC, and Vision

This project targets the core topics of robotics, which includes the following: HRC, Vision, and PbD. During the progress of the investigation, it is expected to gain further insight of Robotics when achieving the stated aim.

1.4.5 To Research About the Current Development of Robotics Related to the Software

In order to build an advanced robot system, existing research on the current development of the ROS and robotics is vital because this could bring new inspirations of approaches and analysis. In particular, the reuse and improvement of existing open-source contribution can be more effective in finishing this project with better quality than expected.

1.5 Research Questions

- Is there any deviation on image recognition each time when performing gripping action, if so, what does the deviation affect?
- How should the planning of task foresee and handle the uncertainties from unexpected human error?
- What would be the critical factor for the tracking of only objects and not humans be improved?
- How does Human-Robot collision affect the overall task procedure?

Chapter 2: Literature Review

2.1 Field of Study

2.1.1 About Programming by Demonstration

Programming by Demonstration (PbD) was conceptualized at the very early time with the statement that a system could perform the record actions again. It allows the system being able to generate programs that correspond to the end user's actions (Cypher, 1994).

Through nearly 40 years of continuous research and development, PbD has become one of the core topics in Robotics and Machine Learning. It is known as a shortcut to extend robot behaviours without massive programming cost. With the application of PbD, it is not required any expert knowledge for a user in robotics or programming skills, instead, the robot would automatically choose its own operation to follow the user according to the its current state generated by the task demonstration.

2.1.2 About Human-Robot Collaboration

Human-Robot Collaboration is a process of that human and robot work together to achieve the same goal, which needs some interaction between human and robot to complete a collaborative task.

Green et al. (2008) mentioned that the importance of making a good HRC:

‘making human-robot collaboration natural and efficient is crucial, in particular, grounding, situational awareness, a common frame of reference and spatial referencing are vital in effective communication and collaboration.’ (p.1)

These conditions (such as situation awareness and spatial referencing) are crucial for a robot system to recognize the role it should play, so it can make the user interact with the robot more intuitively in collaborations

Baraglia et al. (2017) analysed the initiative of robots. They found that in some HRC that requires frequent interaction, or in some HRC requiring frequent

robot assistance/intervention, people would be more likely to accept active robot help. In this paper, the concept of HRC and PbD is a Human-Centred HRC, in which the robot mainly acts as an assistant. In practical applications, especially in industrial applications, HRC is unlikely to make robots actively participate in collaborative tasks due to its lack of perception. Schulz, Kratzer and Toussaint (2018) pointed out these two HRC situations and concluded:

'participants prefer to have control over the interactions for tasks involving joint action and tasks that have higher cognitive load are perceived as easier in robot-led interactions.' (p.10)

The review by Green et al. (2008) also talks 'The simplest way robots can be used is as tools to aid in the completion of physical tasks' (p.3).

2.1.3 Problem Study

Many articles discuss ways to reduce programming costs and simplify the teaching of robot behaviours by showing demonstration actions, and this is known as PbD. For example, a standard welding robotic arm needs to be manually and programmed and configured with different product models. In this case, PbD can make it easier for the robotic arm to execute different poses to achieve reproducible welding, for example, by moving the joints to make the robot end-effector remember the welding positions for a particular product. The demonstration to a robot in these similar applications could be done in many ways, such as moving robot end-effector, moving object, human actions, and so forth. Nevertheless, most of the ways in which robots perform tasks controlled by humans are still set by early complex programming.

CALINON, S. (2019) states that 'Traditional ways of guiding/teleoperating the robot were progressively replaced by more user-friendly interfaces, such as vision' (p.3). A Vision such as using a camera to record images is one of the very typical ways for robots to capture changes in the world. By using vision, when a robot and a human perform the same task, the robot can obtain information conveyed by the human by observing changes in the outside world.

PbD can perform better HRC, because the demonstration of human actions is a more natural way to guide the robot to execute some tasks, so that the robot

can quickly make the next reaction based on given information to follow human changes. The problems raised in this paper revolve around HRC. In this case, the robot must constantly be aware of the way the user works and deal with duplicate actions by considering the advantages of the robot's repeatability.

There are two reasons for trying to experiment with a shared sorting task to make PbD a better HRC:

- Taking the inherent advantage of using mechanics to carry out efficient and accurate physical movements on objects.
- Sorting task is the most typical reflection of automation being used in SMEs.

Granta (2019) has explained some main advantages of robot automation in sorting tasks, such as increased efficiency, lower cost, and improved accuracy. Another article talks about some benefits for a pick-and-place robotic arm, for example, 'robots provide a high level of movement flexibility' and 'Pick and place robots are unaffected by the stresses of the application. They can work without taking breaks or making mistakes' (G.Ugale et al., 2019). These features on stated about Pick-and-Place conclusively proves that robots can do better than human when handling massive repetitive moving action. However, when a human wants to get involved in a cycling task execution, robots are expecting to understand the human actions and treat ones with higher priority.

Prior to the realization of a highly autonomous Internet of Things (regarding Industry 4.0), collaborative tasks were one of the crucial parts of robot applications. Collaborative sorting tasks can play a big role in real industrial jobs. For example, storage robots that handle sorting working in warehouses. People may want to change the storage location of different packages in a flexible manner according to different needs. PbD can help through an external camera sensor, for example by attaching a label containing the new placement information to one of the packages to be sorted to allow the robot to recognize and store all similar packages to the new location, rather than program it. It is complex and time-consuming to programmatically update the location of required packages.

2.2 Related Works

Hazbar Tuly (2019) experimented with human-robot shared tasks, in which human and robot arrange a set of workpieces together. Hazbar Tuly (2019) also states that 'It is important to compensate for the unbalance of capabilities as a first step towards studying and improving any aspect in any HRC scenario', where Hazbar Tuly (2019) thinks human has better perception and dexterity than robots. With taking into account the safety of the human partner in a shared task, task planning includes proactive and reactive decision to avoid hurting the human. So it makes the robot not simply an executor of human commands, but a collaborator.

According to Hazbar's work, we can see that the human-centred HRC can make the robot to give more direct and effective help in cooperative tasks. Because of the guidance of human decision, the robot execution can be adjusted at any time to become more variable, and solve the immediate problems that humans may have.

Goto, H., Miura, J. and Sugiyama, J. (2013) developed a system that could perform a shared assembly task with human partners by identifying the human action to determine the status of the task and generates appropriate assistive actions. During the task, the human could also get some verbal message from the robot to get aware of the robot status. A Mealy Machine has been used to describe the assembly task as it has a high level of complexity. The system is designed to make the robot take over the assembled parts when human needs help.

Goto's system can clearly understand the status of the task by identifying human actions during the process of task execution. The system uses Mealy Machine to divide the assembly task into multiple parts for execution, in order to respond to different reactions from different situations. The state machine seems to be a good solution to describe and manage complex shared tasks, so that the robot can complete the corresponding task more effectively in any situation.

Hamabe t., Goto h., and Miura j. (2015) developed a system that enables the robot to learn from the demonstration of human in a shared assembly task and reproduce the auxiliary process to help human in similar scenes where human needs help. It is worth noting that an FSM can be generated automatically by observing a complete cooperative assembly task, which further improves the robot's ability to understand. Finally, the generated FSM is used to plan the shared assembly task, and the state information of the assembly task is obtained by tracking the actions of people and the colour labels on the workpiece.

Kratzer (2017) studied another similar work, and he or she focused on building a system that allows robots to more successfully complete shared tasks, in which people and robots build bridges by giving clear sharing plans. The shared plan will enable people and robots to have sufficient knowledge not only of the operations performed by themselves but also of the actions performed by their collaborator, so that they can cooperate in any situation. Therefore, the system is not performing a fixed sequence of tasks, but performing different unpredictable subtasks toward the target state. Back to this paper, it is proposed that FSM may be a good solution to execute in different orders to the final state.

Chapter 3: Requirements

3.1 Functional and Non-Functional Requirements

ID	Requirement	Type	Priority
<i>General</i>			
1	The system should ignore and should not follow the mistake made by the user.	Functional	core
2	The system should support updating the task status by subscribing the topic streams.	Functional	core
3	The system should support publishing the task status through topic streams.	Functional	core
4	The system should support the user through one planned sorting task at a time.	Non-functional	core
<i>User</i>			
5	The system should allow the user to have control about when to start the sorting task.	Functional	core
6	The system should allow the user to decide which placement area each type of cube should be placed upon.	Functional	core
7	The system should allow the user to perform a new demonstration anytime they like during the sorting task.	Functional	core
8	The system should enable the user to complete a sorting task by teaching the robotic arm.	Functional	core
9	The system should allow the user to communicate with the robotic arm through an action demonstration.	Functional	core
10	The system should allow the user to demonstrate each type of cube once.	Non-functional	core
<i>Robotic Arm</i>			
11	The system should support the robotic arm for picking up a cube from a 3D position.	Functional	core
12	The system should support the robotic arm by placing a cube through a 3D pose.	Functional	core
13	The system should make the robotic arm return to its 'home' pose after each round of execution.	Functional	core
14	The system should make the robotic arm go to its "home" pose at the beginning of each task.	Functional	core
15	The system should allow the robotic arm to re-plan the execution path at least three times.	Non-functional	core
16	The system should support the robotic arm for execution path planning within 10 seconds.	Non-functional	core
<i>Task Manager</i>			
17	The system should support the task to be paused as long as the user does not give a new demonstration.	Functional	core

18	The system should make the robotic arm complete the sorting of remaining cubes that have the same type in the decided placement area.	Functional	core
19	The system should be able to distinguish the changes in task status.	Functional	core
20	The system should remember the history of the task that it has completed.	Functional (data)	core
21	The system should have a Task Manager object which contains some key information about the task: such as the current status of the waiting area, the aimed status of the waiting status, object poses, the cube ID placed in each placement area, the current status of the placement area, the aimed status of the placement area.	Functional (data)	core
22	The system should have a plan constructed to complete the task within 9 steps.	Non-functional	core
23	The system should support the exchange of information by each state of the task manager.	Non-functional	core
<i>Image Recognition</i>			
24	The system should support a camera sensor to capture object status and to learn the user's demonstration.	Functional	core
25	The system should support pose tracking and estimation of an object.	Functional	core
26	The system should support object recognition to each cube through the ARUCO marker.	Functional	core
27	The system should not fetch the task status (including cube remaining, and placement area status) during pick and place executions.	Functional	core
28	The system should be able to fetch the task status and cube poses up to date when there is no execution.	Functional	core
29	The system should be able to publish the latest image recognition result (including pose estimations, and task information) at least once every second.	Non-functional	core
30	The system should respond to the result of Object Detection within one second.	Non-functional	core
31	The system should support at least a 70% accuracy rate of Marker Detection.	Non-functional	core

3.2 Hardware

3.2.1 Microsoft Kinect RGB-D camera

The Kinect RGB-D camera consists of an RGB camera and an IR depth sensor. Which is used to collect image data for the ROS system along working with the robotic arm.

3.2.2 Universal Robot 5 (UR5)

UR5 is a 6 DOF industrial robotic arm launch by Universal Robots in 2008. According to the international standard ISO 10218 UR5 is a collaborative robot passed with 'EN ISO 13849:2008 PL d' and 'EN ISO 10218-1:2011' standard, Clause 5.4.3 (Universal-robots.com, 2019). This project uses UR5 to perform a shard task with human with Pick-and-Place application.

3.2.3 Tabletop, ARUCO marker board, and ARUCO marker cubes

The task experiments sorting task using 9 cubes labelled with ARUCO makers and 3 ARUCO boards on the tabletop. The tabletop is designed with allowing the robotic arm to manipulate cubes within its working space.

3.3 Software

3.3.1 Ubuntu 16.04

This project implements on Ubuntu operating system 16.04 LTS (Xenial Xerus) that is an operating system based on Linux system. As this project uses kinetic ROS distribution which requires support for Ubuntu 16.04 release (Ros.org, 2019).

3.3.2 Kinetic ROS

Kinetic is a long-term support ROS distribution which is released in 2016. This project requires the Kinetic Kame ROS version to implements the robot system with also some software in particular versions such as Gazebo 7.

3.3.3 MoveIt

MoveIt is a ROS package that provides functionalities for manipulating robots such as motion planning and kinematics, collision check, and so forth. In motion planning, MoveIt is able to show the trajectory of a motion let developers view a preoperative result before executing.

3.3.4 Gazebo 7

This project uses Gazebo 7 to simulate the UR5 for proposed collaborative tasks working together with MoveIt and OpenCV before performing the task on a real UR5 robotic arm.

3.3.5 Python programming language

Python is the main language that is used in implementing this project to produce a designed system, in which python supports a wide library to more efficient programming some features.

3.3.6 OpenCV

OpenCV is an open-source Computer Vision library integrated in ROS, and it is required in this project to deal with images vision that collects from the camera sensor.

Chapter 4: Task Model

4.1 Collaborative Sorting Task

Imagine a smart convenience store, where the placement of items on shelves is executed collaboratively by humans and robotic arms. During the entire placement task, humans decide which items should be placed on which position, and the robot will place the remaining similar products on the shelves in the sequence demonstrated by humans. This proposed imaginary situation will be flexible and efficient. In addition, the placement of different products can be carried out in parallel simultaneously, which allows multiple products to be placed to ensure that each product has the same quantity. The prompt placement system can improve the practicality of the entire placement task, thereby solving the problem where customers sometimes have to wait for their wanted products to be re-stocked.

In our task scenario, we are going to have three types of unique ARUCO coded cubes, and each type has three copies. Human will teach the robotic arm of the target destination of a chosen type of ARUCO cubes where it will be placed. In the process, human picks one cube to one of the ARUCO sorting board to complete the 'teaching'. By detecting and analysing the position of cubes, the change of the number of cubes in placement areas, and the change of number of cubes in waiting area, the robotic arm should be able to understand and foresee its targets and start the sorting. The sorting task is completed in sequence through a dynamic planned queue. The dynamic planning queue will be re-scheduled every time the task flow changes; for instance, when the human shows another demonstration right after the first one with different types of cubes and moving it to a new location, thus, the plan changes. The cubes should be sorted in a dynamic sequence as it allows a uniform distribution of task execution on all cubes regardless of the sequence of demonstration in the plan. Moreover, the detecting is continuous, which means any further human action will be re-planned to a high level of priority and will be executed immediately. Last but not least, the task will end until all the cubes have been allocated.

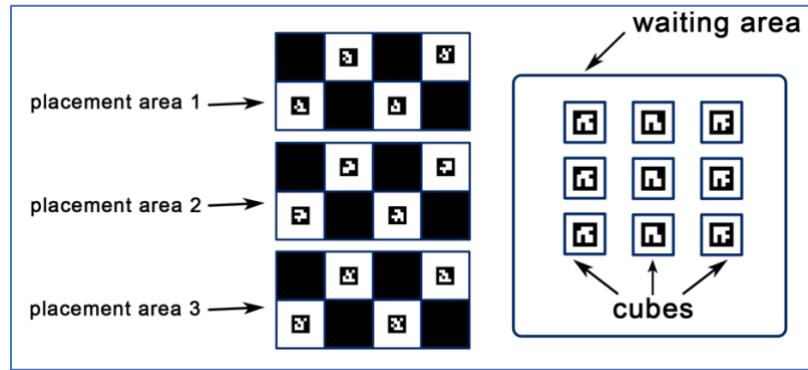


Figure 1. The Layout of the testbed for Implementing the Sorting Task

This collaborative task is simulated on a working platform (the testbed on the QMUL metallic frame) with four labelled areas, with Figure 1 showing the layout of the working platform of the experiment. One of the areas (we call it the “waiting area”) is used for the placement of unsorted cubes, and the rest areas (we call it the “placement areas”) are for allocating the sorted cubes. The “placement areas” will be 2D code boards in which each sorting subarea is coded uniquely. We are going to use three types of unique ARUCO cubes, and each type will have three copies. The task is to sort these coded cubes into different “placement areas” starting with human demonstration, and the task ends when no cubes exist in the “waiting area”.

4.2 State Machine

This section details the State Transition Model to describe the sequence of the task that will be performed by the robotic arm and the user.

4.2.1 State Definition

There are two types of states designed to achieve the task: 1) Demonstration State; 2) Execution State. Both types of the states contain some necessary information about the task, while some information is different.

For the Demonstration states:

- The specified id (marker ID) of the demonstrated placement area for a cube.
- the status of all placement areas (the capacity).
- The remaining number of the cube types (ID)

For the Execution states:

- The pose of the targeted cube object.
- The pose of the targeted placement.
- The type (ID) of cube currently under execution.
- Whether the user makes a new demonstration.

For Both States:

- The status of placement areas.
- The cube poses.
- Next state to go.

4.2.2 Task Clarification

- The movement of a cube demonstration is from the “waiting area” to the specified “placement subarea”.
- The human demonstration (the “placement area”) tells the current sorting cube where it would be placed.
- State transitions are representations of the decisions made connecting to the next possible state.

4.2.3 State Descriptor

- **(D_a)**: The letter “D” refers to “Demonstration” and “a” refers to the type of cube.
- **(M_a_2)**: The letter “M” represents the execution on the **second** “a” type of cube.

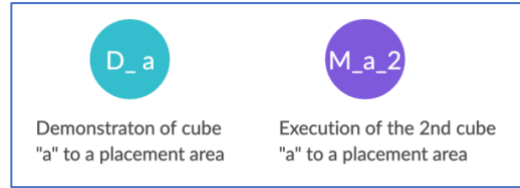


Figure 2. The description of two samples of the graphical states

The Task model has two kinds of states involved, where one of them is a static state and another one is a dynamic state. Figure 2 shows the graphical state corresponding to the state descriptors described above. The state with dark (purple) background refers to the execution of the robotic arm and. The state having light (blue/green) background refers to the demonstration of the user to a cube. Moreover, during these states with a dark background, in some cases, the robotic arm might need to wait for a human demonstration to go to the next state. So, the task can be discontinuous in some states and the states may track the historical information of the finished classification and the status of the placement area as conditions of the next possible state.

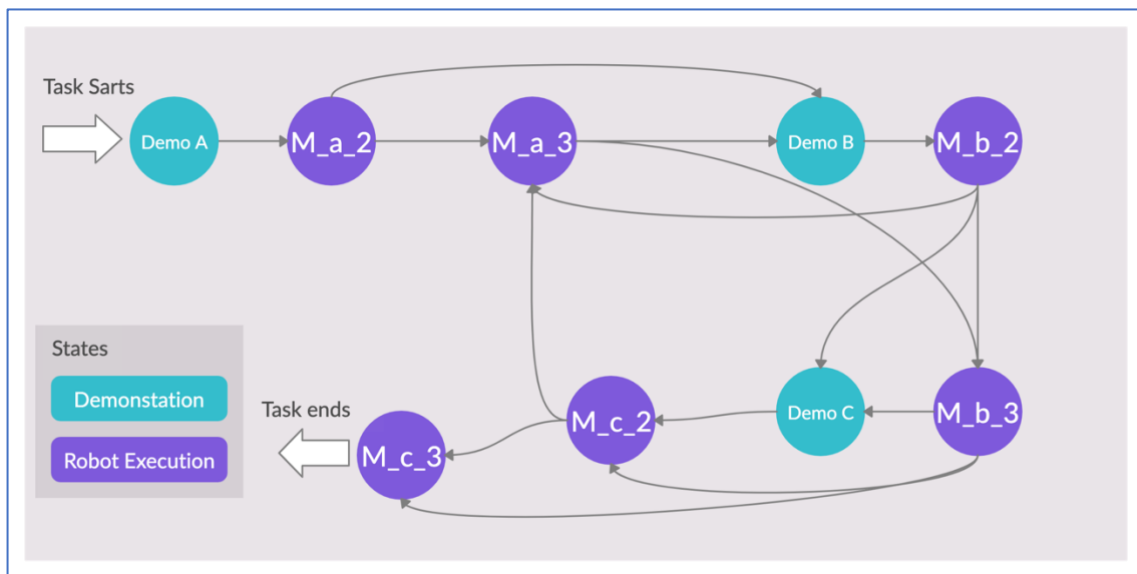


Figure 3. The State Transition Model of the collaborative task

Figure 3 shows the task model starts from nine remaining cubes waiting to be sorted, and finally reaches zero remaining cubes. The transition between the states is the decision of how a task will be done in a different sequence of sorting. The decision is made by capturing a new demonstration of another cube, which can change the next step of the execution task of the robotic arm. It should be noted that "a", "b", and "c" represent the first, second, and third sorted cube. This means that regardless of the actual type the ARUCO code represents, the order of execution and demonstration will define the sequence by "a", "b", and "c". In fact, if we choose to declare the actual cube type and placement area, this sorting task will have several State Machine units working together. Figure 4 below shows the different groups of state machines when we switch to the actual cube type, and there are six combinations in total for three types of cubes. In contrast to the aforementioned State Transition Model, this performs the same work, but the actual cube type must be identified to start with different state machines. It can be seen that the 9-state FSM can compute many different sequences of sorting, in fact, there are more than 1000 sorting sequences can be executed based on different cases of user demonstrations.

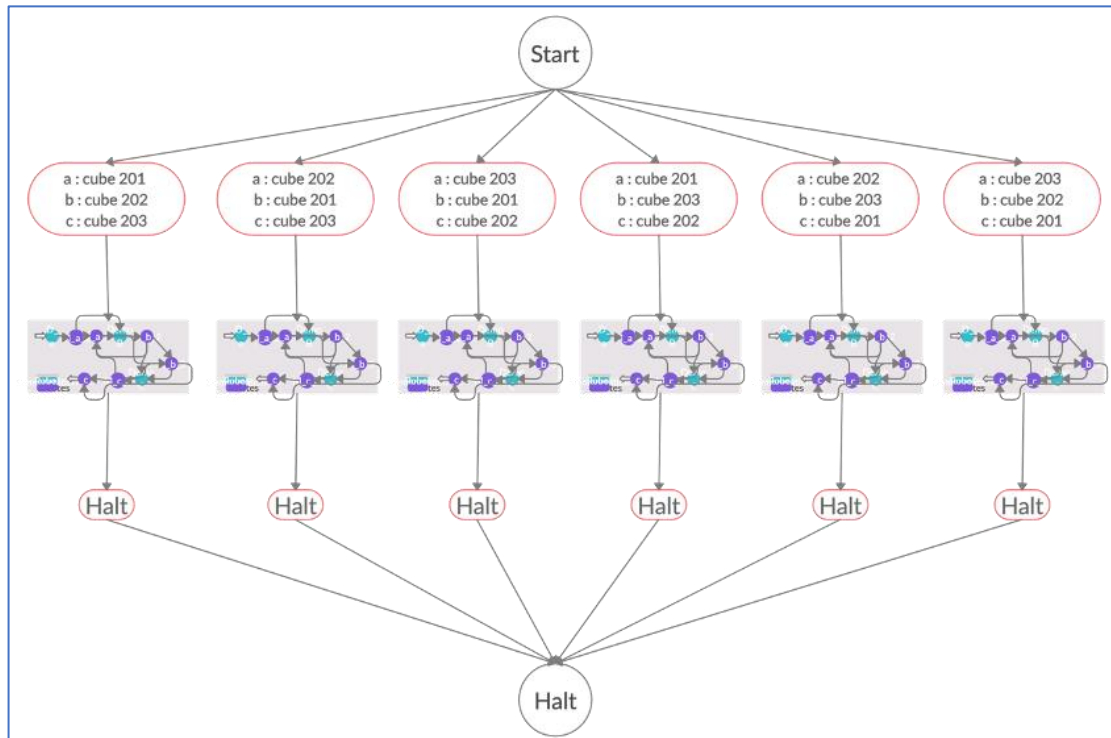


Figure 4. The State Machine Model When Cube Types Are Specified

Chapter 5: Implementation

5.1 Computer Vision

Computer Vision is crucial in this system. It mainly completes the job by two tasks:

1. Calculating the pixel position by identifying the ARUCO marker ID
2. Estimating the 3D pose from the pixel position and the depth measured by the depth sensor.

Several important image recognition information is published in real-time through the ROS framework, which includes the following:

<i>Information</i>	In the Waiting Area	In the Placement area
<i>3D Pose of the cubes</i>	Yes	No
<i>Marker ID of each cube</i>	Yes	Yes
<i>Number of cubes</i>	Yes	Yes
<i>Marker ID of area itself</i>	No	Yes
<i>3D Pose of the area itself</i>	No	Yes

5.1.1 Object Detection

This system takes advantage of the powerful OpenCV for both Image Processing and Image Recognition. OpenCV supports ARUCO marker recognition as a built-in library with lots of fully integrated methods such as marker creation and marker detection. With the extensibility of the ROS, it is straightforward and simple to construct a “bridge” to connect external libraries like OpenCV. And the transformation of Image Streaming message in ROS topics to the OpenCV Image Object works seamlessly.

5.1.2 Pose estimation

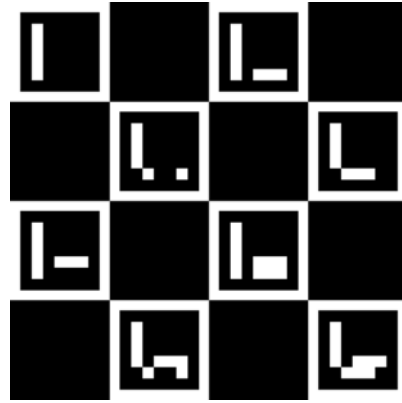


Figure 5. The ARUCO Board for Camera Calibration

Regarding the very early processes of Pose Estimation, it is more accurate to use a camera calibration to map the 3D world to a 2D image having using the camera calibration. It helps in estimating the parameter of the camera, especially the camera matrix. To further understand the role of Camera Calibration, one sample is to place the ARUCO board (see Figure 5) generated by the system in the upper right corner of the working platform, and the example in the camera frame has been shown in Figure 6. After reading all the samples, the system is able to obtain corrected images (see Figure 7) through camera distortion and camera matrix.

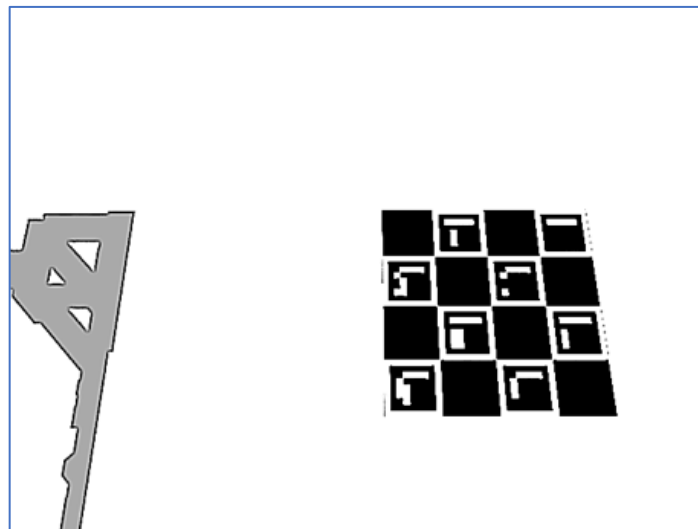


Figure 6. One Sample of the Camera Calibration

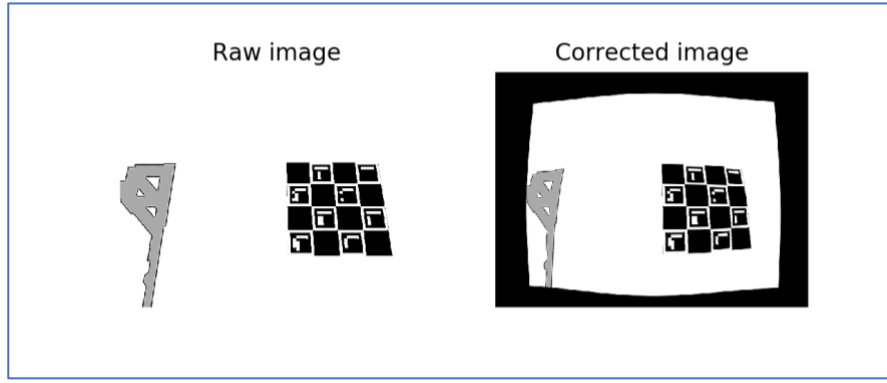


Figure 7. Corrected Image with Camera Matrix

Furthermore, through the further use of the detected pixel position, the system can implement operations to convert 2D pixel positions and 3D poses. Although OpenCV provides the methods to estimate the pose of the ARUCO marker from 2D images, which has shown in Figure 8, there is a certain discrepancy with the actual pose. Alternatively, take advantage of the depth sensor of the Kinect Camera. The computation from 2D image pixel position to 3D poses can be done manually. The computation uses the reference to the camera's 3D spatial position and Euler angle information. It calculates the trigonometric functions and quaternion conversion to finally obtain the 3D pose of the object, in order to realise the grasping of the object by the robotic arm.

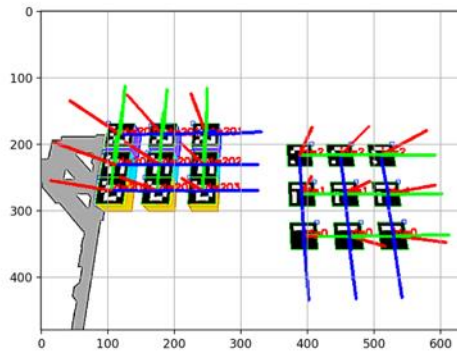


Figure 8. Marker Detection in OpenCV

5.1.3 Image Processing

The above object detection solution can complete the designed sorting task in most of the cases, but the accuracy of the marker detection rate is relatively low. Consequently, some cubes in the waiting area have a chance to be unrecognised. And according to some experiments, the task will not be completed; for example, when one of the cubes cannot be recognised, there will be conflicts

between the actual status and the target status. Eventually, the finite state machine will be processed into a standby state under error conditions.

In order to complete the task more accurately, the system adopts the filtering technology widely used in digital image processing. The idea of applying filters is to make the image in high contrast and thus, make the mark easier to recognize. Because of the camera resolution is relatively low, the details of some markers are difficult to capture. The system first applies Gaussian smoothing with the mask kernel size 5, and then uses two 8-neighbouring Laplacian enhancements to obtain the sharpening image. Finally, the image is converted into a binary image by adaptive thresholding with kernel size 5 and block size 17. The filtered image can significantly improve the mark detection and the stability of the whole system.

5.2 Robot Control

The control of the robotic arm in this system is prototyped through the Python-based Move Group Interface, which is considered as simpler and faster to implement than using C++. MoveIt! Move Group is an independent and complete package that is stand-alone with ROS, and It contains many essential operations about moving a robot, such as a move robotic arm to a defined joint goal. Similarly, this system uses Cartesian Path planning combined with multiple waypoints on each path execution, and the performance is reasonably good. However, some planned paths are not optimal and are extraordinary from a human perspective. Another feature that is worth mentioning is the avoidance of the collision to scenes. The planned path avoids the collision to the scenes added in the RVIZ model, which means that it could identify the obstacles such as the QMUL metallic frame, and the cubes (or boxes). With the Gazebo plugins like Grasp-Attach Plugin, the robotic arm can pick an object by attaching the object model to the robot link in Gazebo simulation.

5.3 Task Manager

The core of completing complex tasks is using FSM to act as a Task Manager. FSM controls the flow of the task operation and different progress of the task in each situation. Using SMACH library in ROS, it is efficient to plan the task by creating different states and connections using defined logical expressions. Moreover, different executions can be added in different states when needed; for example, the robotic arm picks cube A in Cube A states when goal status is fulfilled. The designed FSM improves the optimisation of the sequence of tasks while ensuring the completion of tasks. Therefore, the system could understand all cases of user demonstrations at any time.

Chapter 6: Presentation

6.1 Fragment of Code for SMACH

The following is a fragment of code to implement SMACH from FSM using python language. By using SMACH we can easily transfer the state transition model to machine language to plan tasks for a robot. The code below creates a State using SMACH.State Class to execute the second “a” type cube into placement area.

```
class Move_a2(smach.State):
    def __init__(self):
        smach.State.__init__(self, outcomes=['out1','out2'])
    def execute(self, userdata):
        rospy.loginfo('Executing state Demo_a')

        task.cube_poses = robot_move.cube_poses
        try_count = 0
        sub_changed = False

        cube_id = task.get_cube_id('a')
        sub_id = task.get_sub_id('a')

        while sub_changed != True:
            if try_count == 3:
                break
            try_count += 1
            rospy.loginfo("SUB NOT CHANGED!")

            if task.cube_poses[task.get_cube_ind(cube_id)] != []:
                x,y = task.cube_poses[task.get_cube_ind(cube_id)][0]

                robot_move.do_pick(x,y)
                robot_move.set_target(sub_id)

                robot_move.do_place()
                rospy.sleep(0.5)

                task.status[0] = robot_move.sub_status

                sub_changed =
                    task.check_sub_changed(task.status, sub_id)
            task.cube_poses = task.cube_poses[1:]
            task.move('a')
            print(task)

        sub_changed = task.check_sub_changed(task.status, sub_id)
        if sub_changed == False:
            rospy.loginfo("no more demonstration")
            return 'out1'
        else:
            rospy.loginfo("demonstration: {}".format(sub_changed))
            return 'out2'
```

Chapter 7: **Evaluation**

This section analyses the overall findings and results of the evaluation in hand. Due to some limitations imposed on testing the system, and the lack of Simulator, some aspects of this project were not ascertainable. And because of the Covid-19 pandemic and the government's Social Distancing policy, the evaluation is only taken from the participation of the developer and the primary user without other stakeholders and communities joining.

7.1 System Findings

7.1.1 Achievement

According to the Project Aim in Chapter 1, this system can fulfil what the system should be behaved and can be managed to complete the proposed task except testing the system on the real UR5 in ARQ Laboratory. The sudden Pandemic and University Lockdown lead to the cancellation of the plan on implementing the system compatible with the physical equipment like the real external camera sensor and the real robotic arm. But the main concepts and objectives are achieved and researched with the system; as a result, this project engages the topics of PbD and HRC using various knowledge gained from Computer Science and Robotics.

7.1.2 Qualitative Results

Concentrating on the Task Scenario and Task Modelling in Chapter 5, this project created a corresponding FSM to represent and plan the task. From the perspective of understanding the problems and solving uncertainty and flexibility, thinking the way as FSM allows us to consider all potential problem that may exist in the task, and all the possible way to complete the task. The State Machines have representations of all the possible task flows; thus, the execution speed and response speed of which is much faster than the human brain. Tic-Tac-Toe is a similar task in the Artificial Intelligent Game Theory, which can compute the possible paths of states that are with the highest benefits to beat human beings. However, in this project, our scenario is to help the user having control of the

sequence of the task. Both are designed to handle dynamic input and always provide the same output at the end.

Apart from completing tasks, FSM will respond regardless of whether the ROS system is idle or busy with the execution of the robot. In particular, FSM has stable responsive to ROS immediately after the execution of the Move Group to ensure that the system is running under management. Therefore, when controlling the robotic arm of the task, it will never miss the communication with the user, and it allows the Task Manager ready to listen at any time.

In addition, FSM, as a task manager, supports modular system development and systematic problem-solving. It is also cheap to change only the FSM part of the system to allow the system to operate in different required ways. Another advantage of the state transition model is the ability to transform complex tasks into very simple sequential datasets so that the logic fault can be debugged quickly.

According to the FSM designed in Chapter 4, the system can perform tasks in different sequences with a fixed final state (status). In other words, once the type of cube (marker ID) is decided, no matter which cube the user decides to demonstrate first, and when the user decides to start each demonstration, the final placements of all cubes are the same.

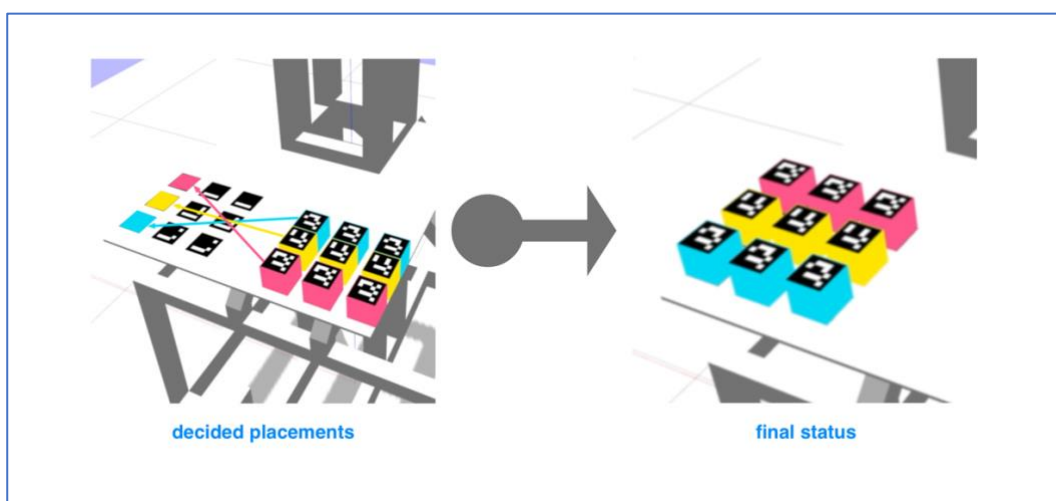


Figure 9. The Decided Placement Area and The Final Status of The Placement Areas

In one of the experiments performed, the user decided to place the red cube in the placement area 1, the yellow cube in area 2, and the blue cube in area 3. In Figure 9 above, the left image shows the placement area for each type of cube, and the right image shows the final status of the placement area when the task is completed. The three sequences for completing tasks are:

1. The user demonstrates a red cube and waits for the robot to execute the remaining red cubes, then demonstrates a yellow cube and waits for the robot to execute the remaining yellow cubes, and finally a blue cube and waits for the robot to complete the task. Figure 10 below shows the FSM path and the progress of this sequence in the Gazebo simulator.
2. The user first demonstrates a red cube, then demonstrates the yellow cube when the robot executes the remaining red cubes, and finally demonstrates the blue cube after all the red and yellow cubes are sorted and waits for the robot to complete the task. Figure 11 below shows the FSM path and the progress of this sequence in the Gazebo simulator.
3. The user demonstrates a red cube, then immediately demonstrates a yellow cube and then blue, and waits for the robot to complete the task. Figure 12 below shows the FSM path and the progress of this sequence in the Gazebo simulator.



Figure 10. The Sequence of Example 1 and The FSM Path



Figure 11. The Sequence of Example 2 and The FSM Path

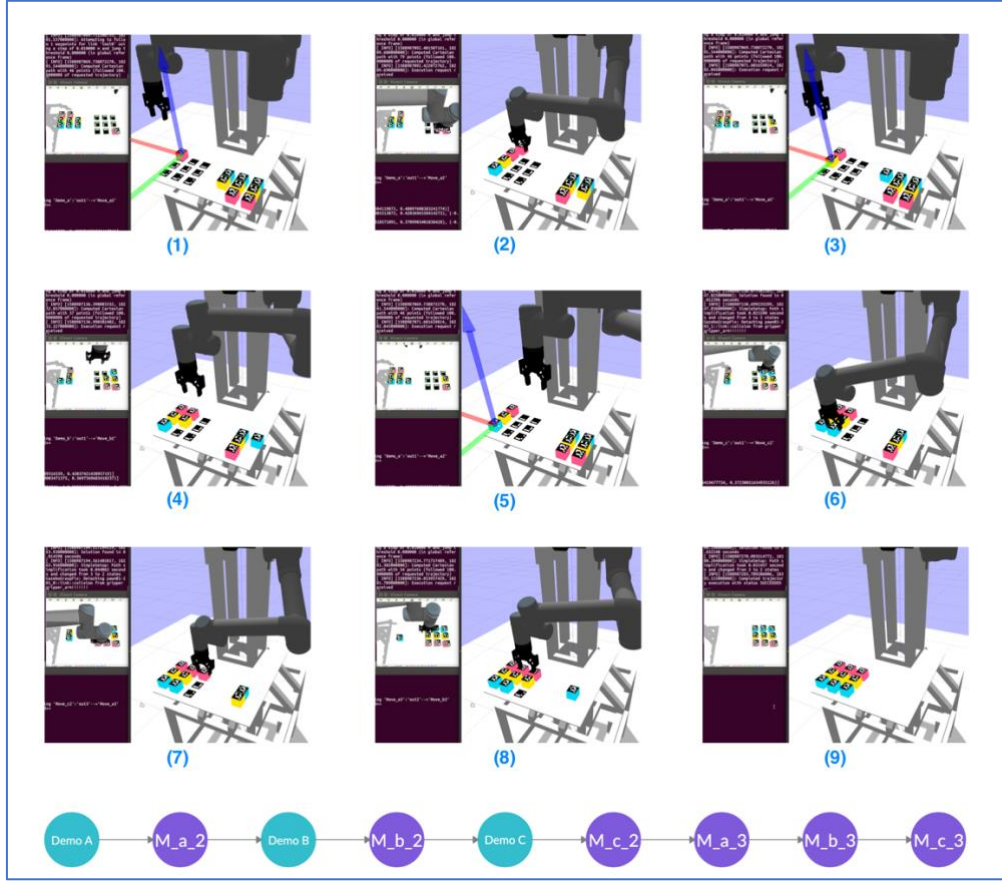


Figure 12. The Sequence of Example 3 and The FSM Path

7.1.3 Quantitative Results

Another equal significance is to evaluate the system on how “success” it works. In order to improve the capacity of the system, what constitutes the “success” are effectively obtaining from different parts of the system. To be more precise, Image Recognition affects the accuracy and speed of sorting a cube in our scenario. Before applying Image Processing techniques, the system has chances to fail to place a cube because of the wrong pose feedbacks, thereby, leading to a failure on image recognition when the placed cube is too far from the placement area to be recorded.

We record the correctness of the cube detection by randomly placing the cube 100 times on the working platform and repeat this test 9 rounds. The number of cubes gradually increased from 1 in the first round to 9 in the ninth round, in order to detect the accuracy of detection in different numbers of cubes.

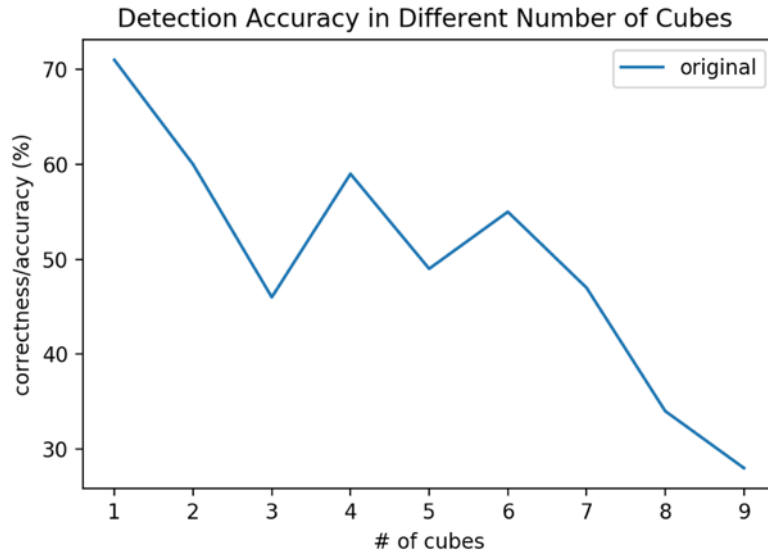


Figure 13. Detection Accuracy without Image Processing

Figure 13 shows the testing of accuracy on the original Image Detection, and we can see that the detection success rate is about 72% when there is only one cube, and only 30% when there are 9 cubes. This means that as the task progresses, the detection success rate will become higher and higher as the number of remaining cubes decreases. Whether the task can be completed usually depends on the success of the detection of the early task. If the detection of the early task is wrong, it will directly affect the correct operation of the entire State Machine.

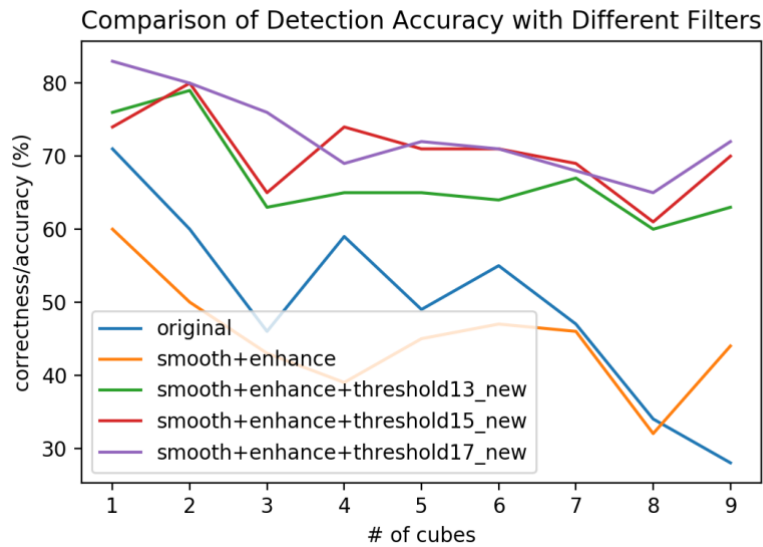


Figure 14. Comparison of Different Image Processing Filters

During the testing of several methods of Filtering combinations, it is noticed that applying Smoothing and then Enhancement, finally Thresholding with block size 17 distributes the highest accuracy in Marker Detection. As we can see in Figure 14, the purple curve has the best performance of Detection, in which the accuracy is about 83% when there is one cube remaining, and is around 75% when 9 cubes remaining. The blue curve shows the accuracy rate from one to nine cubes remained without any filtering, and there are vast differences in accuracy when having different remaining cubes. In contrast, the curves after applying Image Processing are more flat, which means the accuracy rate is relatively stable when dealing with different remaining cubes.

7.2 Comparison to the Gaps

The main purpose of this project is to use FSM to build a suitable state transition model and convert it to SMACH to be compatible with ROS. Although the early State Transition Model designed was not able to be converted to SMACH, the current simplified version can complete the implementation successfully. This project has also conducted research on the theme of robotics, and comprehensively utilised knowledge in multiple fields such as PbD, HRC, computer vision, image processing, FSM, besides, it has used some popular tools of Robotics to bring up all together to complete the proposal, such as ROS framework, Gazebo and MoveIt. By studying the ARUCO marker detection algorithm in OpenCV, the system has obtained benefits of robust and simple from ARUCO in the initial prototype. Nevertheless, the project also helps gaining a wealth of experience in controlling the Industrial Robotic Arm UR5 despite facing many challenges. The progress of system programming starts from learning all the basic knowledge of robot technology, which is also a good start for future investigation.

7.3 Limitations

This system achieves what the scenario task proposed, but it still has much shortage in real practices. This includes some problem with lack of professional in several aspects, for example, the technique of image recognition is the lack of in-depth and advanced, and the control of the robot is too simple. The following points are about the limitations of the current work and are needed to dig deeper into topics in future work.

1. It supports limited picking and placing actions and angles by the robotic arm.
2. The planned path of the robotic arm is not optimal, and it takes longer time than expected.
3. The Object Detection and Pose Estimation are estimated to be about 0.5 seconds. Although it meets the requirements of our design, it is observed that there are sometimes delays in performing tasks.
4. It is not supporting human body recognition, therefore, lack of avoidance of collisions.
5. The Finite State Machine is not extensible; therefore, it is only suitable for well-specified and similar tasks.
6. It does not provide a solution to the mistakes made by the user.

Chapter 8: Conclusion

This section includes the discussion about the achievements and challenges faced when developing the proposed system. Accordingly, some future work that deals with the evaluated limitations also states here.

8.1 Achievements and Challenges

This project allows me to apply my knowledge gained from a Computer Science background, and also learn new topics in different domains. Likewise, this project allows me to control over process of Robotic Software Development and tackle problems phase by phase. Throughout the project, my personal interest of studying Robotics in the future has become stronger. Although some basic algorithms are used in the current prototype implementation implement, the system can complete the expected work and can meet all the core requirements designed in Chapter 3.

However, this system is only implemented by configuring the UR5 and a Kinect RGB-D camera in the Gazebo simulator, where the program does not support the interface that allows operations on a real robotic arm UR5. It is worth noting that there may be some potential problems may exist when the system is tested in the real-world environment, including image noises from the external camera sensor, mechanical tolerances of the joints on the UR5, and grasp friction effected on the gripper. All of these uncertainties from real-world experiments may lead to a completely different evaluation result of the system.

At the same time, I have also encountered many challenges in Robot Control, and most of which was spent learning to control a robotic arm. This is mainly because I have no basic knowledge in this field, thus, my learning cost is relatively high. For example, in the process of understanding the structure of a robot and the trajectory of a robot action, I need to learn some principles of multi-joint kinetics and the mathematical rules of 3D orientation and rotation. In addition, in the process of learning the ROS framework, there are also many technical problems which I have to figure out by myself. One problem that has troubled me

for a long time is the wrong controller configuration of the gripper in MoveIt, which made me stuck for a while at the beginning of this project.

In addition to expensive learning costs and lack of basic knowledge, another challenge is the limitations of object recognition. This is because the object-oriented image recognition system cannot capture the motion of the human body, therefore, a collision between the human and the robotic arm may occur when performing a collaborative task. However, since the experiment can only be carried out in the simulator, this problem is unsolvable in this project, thus, it is not possible to further develop collision protection for the human body.

8.2 Future work

Here are some ideas about the future work, which are mainly aimed to deal with the shortage listed in Chapter 7. The future work on the one hand improves the degree of completion in terms of the aim and objectives, on another hand makes the current prototype more functional.

Concentrating on making the system more functional, the system must be more advanced in controlling the robot with more actions. It is clear that the current picking and placing actions are too limited to cope with more complex situations, particularly, it is not able to grip the objects when objects are sticking too close. So, it is required a method that can find out a best picking angle for the robotic arm to have problem-solving ability. Apart from smart picking and placing, avoiding obstacle collision is also an important feature, which could help completing some custom trajectory. By recognize the obstacles, the robotic arm could be able to execution some task in narrow workspace. For example, putting a stick through a round hole, where it must detection the object figure from 3D static collisions. The end-effector must be controlled very carefully, which need to be used to deal with many cases in the real world. For example, if the box that the robotic arm picks is not allowed to be turned over, because the items in it are easily damaged. In this case, we must keep the top of the box always facing upward, but the current path planning method cannot satisfy such execution.

Furthermore, the future work would should also focus on emergency pausing from predicting 3D collisions, task training for identifying the mistakes, and task planning for more complex tasks.

Concentrating on handling user mistakes, the future work may consider training a neural network to identify the accidents like a human brain, therefore, it can find out solutions more precisely. For example, if the user gives a confusing wrong demonstration, the robotic arm should be able to think as a human and distinguish it between similar mistakes from the trained network. Last but not the least, future working may implement human body detection algorithm to avoid Human-Robot collision, which is particularly vital regarding human security.

Chapter 9: Risk Assessment

Risks	Likelihood	Severity	Impact	Preventative / Mitigating Action
Poor time management	Low	High	- Lack of quality - Unfinished system / project	- The project needs to follow the project plan and spend more time on it. - Evaluate progress frequently and adjust time plan if necessary
Constant change in project plan leading to failure	Medium	High	- Project failure - Missing the original target	Keep the original target and ensure the changes do not affect the main purpose (e.g. the task modelling methods)
Acquiring new skill leading to project delays	High	High	- Constant change in time plan - Not delivering on time - Low quality	- Avoiding poor time management since it directly affects time spent on the project. - Learn new skills reasonably
Change the topic of the project / change the project	Low	Very High	- Low quality - Not delivered on time	- In-depth literature research and study - Avoid stopping the project mid-stream
Design/task does not fit the aim	Low	Medium	The system misses some expected solutions to problem stated	Adjust the design as necessary according to the project objectives and aims
Problems in communication with supervisor	Low	Medium	Low quality	Communicate ideas and project vision clearly and effectively

			• Not on a right direction • Miscommunication could lead to project delays	• If communication fails, try to adapt while considering the project goals
Burnout caused by overworking or stress	Medium	High	• Affects the health and mental state • Cause delays • Low quality	• Effectively manage project and time. Break the task down into parts and work on them one by one
Hardware failure	Medium	High	• Loss of data/work • Not delivered on time • Unfinished project	• Try to conduct backups and work on the project in a way that is compatible with the available hardware.
Aim too high/plan too much	Medium	High	• Not finished • Not delivered on time • Low quality • Missing the aims and problem solution	• Use open-source achievements to aid and focus on the main objective and topic
Algorithm is too hard to implement	Medium	Medium	• Low quality • Missing solution to the problem stated	• Do not spend too much time on doing unsolvable implementation, try to find alternative solution (avoiding P=NP)

References

Baraglia, J., Cakmak, M., Nagai, Y., Rao, R. and Asada, M. (2017). *Efficient human-robot collaboration: When should a robot take initiative?*. The International Journal of Robotics Research, 36(5-7), pp.563-579.

Calinon, S. (2009). *Robot programming by demonstration*. Lausanne: EPFL Press.

Cypher, A. (1994). *Watch what I do*. Cambridge, Mass.: The MIT Press.

G.Ugale, Y., R. Ugale, G., B. Dhote, A., S.Dawange, S., G.Vase, N., R.Ghodke, S. and S.Wankhede, P. (2019). Design & Analysis of Pick and Place Robot. *International Journal of Scientific Research in Science, Engineering and Technology*, 4(4), p.993.

Goto, H., Miura, J. and Sugiyama, J., 2013. Human-robot collaborative assembly by on-line human action recognition based on an FSM task model. In *Human-robot interaction 2013 workshop on collaborative manipulation*.

Granta-automation.co.uk. (2019). *What are The Advantages of Automatic Product Sorting?* | Granta Automation. [online] Available at: <https://www.granta-automation.co.uk/news/what-are-the-advantages-of-automatic-product-sorting/> [Accessed 15 Nov. 2019].

Green, S., Billinghamurst, M., Chen, X. and Chase, J. (2008). *Human-Robot Collaboration: A Literature Review and Augmented Reality Approach in Design*. International Journal of Advanced Robotic Systems, 5(1), p.1.

Hamabe, T., Goto, H. and Miura, J., 2015, December. *A programming by demonstration system for human-robot collaborative assembly tasks*. In *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)* (pp. 1195-1201). IEEE.

Hazbar, Tuly, (2019). *Task Planning and Execution for Human Robot Team Performing a Shared Task in a Shared Workspace*. Thesis. Rochester Institute of Technology.

J. Zhang, Y. Wang and R. Xiong, 2016, "*Industrial robot programming by demonstration*" 2016 *International Conference on Advanced Robotics and Mechatronics (ICARM)*, Macau, pp. 300-305.

Kratzer, P., 2017. *Robot Assistance for Collaborative Task Execution*. Postgraduate. University of Stuttgart.

PricewaterhouseCoopers 2014, *The new hire: how a new generation of robots is transforming manufacturing*, PricewaterhouseCoopers, [New York], viewed 22 Nov 2019, <<https://www.pwc.com/us/en/industrial-products/assets/industrial-robot-trends-in-manufacturing-report.pdf>>.

Rane, D.B., Sagar, S., Devendra, V., & Jameer, U.S. (2015). *Automation of Object Sorting Using an Industrial Roboarm and MATLAB Based Image Processing*.

Ros.org. (2019). *REP 3 -- Target Platforms (ROS.org)*. [online] Available at: <https://www.ros.org/reps/rep-0003.html#kinetic-kame-may-2016-may-2021> [Accessed 21 Nov. 2019].

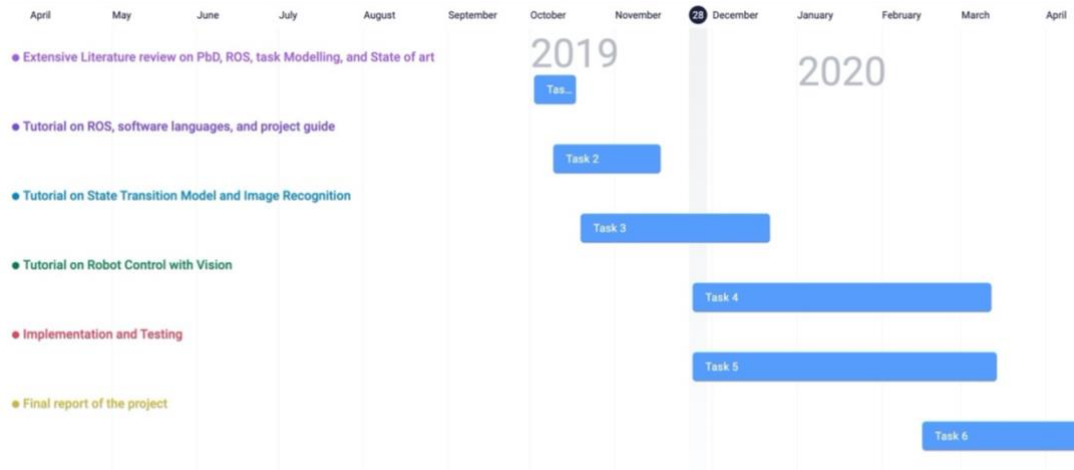
Roy, D., Hsiao, K. and Mavridis, N. (2004). *Mental Imagery for a Conversational Robot*. IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics), 34(3), pp.1374-1383.

Schulz, R., Kratzer, P. and Toussaint, M. (2018). *Preferred Interaction Styles for Human-Robot Collaboration Vary Over Tasks With Different Action Types*. Frontiers in Neurorobotics, 12.

Universal-robots.com. (2019). [online] Available at: https://www.universal-robots.com/media/50588/ur5_en.pdf [Accessed 21 Nov. 2019].

Appendixes

Project Plan



Milestones

Milestones	Description	Completion Date
Problem Identify	The problem found that motivates to do about this project.	9 Nov 2019
Literature Review	Research on topics: 1. Programming by Demonstration 2. Human-Robot Collaboration 3. Pick and Place	28 Nov 2019
Learn image recognition with OpenCV and Pick and Place	This includes recognising the object, tracking the pose, and calculating the 3D position using image processing and computer vision technics in ROS.	29 Dec 2019
Task Model Design and Implementation	Constructing a task model in ROS to execute the collaborative task.	10 Jan 2020
Experiment with MoveIt and gazebo	This is to test the functionality of task executing and robotic arm controlling.	28 Jan 2020
Simplified State Machine Model for Task Execution	Experimenting with different tasks with different sorting sequences.	20 March 2020
Final Report	Constructing a report containing all above.	20 April 2020